Informe 3 · Análisis de tráfico

Sistema integrado de eventos laborales (JML) · Andina Servicios S.A.S. (empresa ficticia, datos ficticios)
Politécnico Grancolombiano · Gerencia de Proyectos en Seguridad de la Información · Grupo D01
Auditoría del 24 al 25 de octubre de 2026 · rama `auditoria/entrega` · commit `c6e6490` · generado el 25/10/2026

1. Resumen

Se capturó el tráfico del stack local en **tres puntos** mientras corrían los flujos reales del sistema, y se analizó con `tshark` dentro de una imagen fijada por digest. Cada captura lleva **aserciones**: no se informa lo que se vio, se informa si lo esperado se cumple.

Propiedad	Esperado	Medido	Resultado
El tráfico externo viaja cifrado	TLS 1.3, una suite AEAD	10 de 10 saludos con TLS 1.3 (`0x0304`) y suite `0x1302` (TLS_AES_256_GCM_SHA384)	**Cumple**
El campo restringido no llega a GLPI (T7)	0 tramas con el centinela hacia GLPI	0 de 31 240 (flujos RET y EMG sobre GLPI 11 real) y 0 en el tramo WAF->GLPI	**Cumple**
El campo restringido no sale del equipo en claro	0 tramas en el tramo externo	0	**Cumple**
Las contraseñas no viajan en claro hacia la base	0 mensajes de autenticación en claro	0 (21 negociaciones SCRAM)	**Cumple**
Los tokens de GLPI no viajan en claro (N-1)	0	**382** `App-Token`, **378** `Session-Token`, **4** `user_token`	**No cumple** -> B-32
GLPI cifra su tramo con MariaDB (V-04)	TLS	30 133 tramas, **0** con TLS, 11 051 consultas legibles	**No cumple** -> B-30
Las consultas a la base van con parámetros	Protocolo extendido	0 `Parse`/`Bind`, 1 332 consultas simples	**Matiz** -> §6

Los `.pcap` **no se versionan**: contienen tokens y valores centinela. Lo que queda en el repositorio es el comando exacto, la salida del análisis (con los secretos enmascarados) y el hash de cada archivo.

2. Método

- **Captura**: `tcpdump` de la imagen `nicolaka/netshoot` (digest `b09d9b21`), adosada al **espacio de red de cada contenedor** (`--net container:<nombre>`), con las capacidades `NET_RAW` y `NET_ADMIN` solo para esa captura.
- **Puntos**: `jml-bunkerweb-1` (cliente?WAF por TLS y WAF?web por HTTP), `jml-postgres-1` (aplicación?base) y `jml-glpi-1` (worker?GLPI y GLPI?MariaDB).
- **Flujos**: los de verdad. Para GLPI, `contrato-glpi.sh` con RET y EMG sobre GLPI 11 real. Para el borde y la base, el recorrido Playwright por el WAF (ING con PDF, VAC, RET y EMG).
- **Centinelas**: valores que no existen en ningún dato real (`SENTINELA-RESTRINGIDA-9f3a`, `CENTINELA-INTERFAZ-<marca>`, `CENTINELA-EMG-<marca>`) escritos en los campos restringidos. Si aparecen donde no deben, es una fuga.
- **Neutralización**: antes de cada captura, copia de la base y baneos levantados; después, restauración (las filas vuelven a 414/21/88/75).
- **Control positivo**: cada aserción de «0 tramas» está acompañada de una de «más de 0 tramas» en el punto donde el valor sí debe estar (la base y el tramo interno WAF->web). Una captura que no ve nada no prueba nada.

Scripts: `scripts/captura-trafico.sh` (GLPI) y `scripts/captura-trafico-web.sh` (BunkerWeb y PostgreSQL).
Evidencia: `evidencias/trafico/F7-04-CAPTURA-TRAFICO` y `evidencias/trafico/ENTREGA3-CAPTURA-WEB-Y-POSTGRES`.

3. Tramo externo: cliente ? WAF

Medida	Valor
Tramas capturadas en `bunkerweb`	821
Saludos del cliente que ofrecen TLS 1.3	10
Versión que fija el servidor	`0x0304` (TLS 1.3) en los 10

Medida	Valor
Suite que fija el servidor	`0x1302` (TLS_AES_256_GCM_SHA384) en los 10
Tramas con el centinela en el tramo cifrado	**0**

El borde ofrece solo TLS 1.2 y 1.3 (`testssl.sh`: sin SSLv2/3, TLS 1.0 ni 1.1; HSTS de un año; sin Heartbleed, ROBOT, POODLE, BEAST, SWEET32 ni RC4). La redirección del puerto 80 a HTTPS se midió aparte en la matriz de puertos (301). **Cumple RS-01 y descarta T19 en este tramo.** El certificado local es autofirmado (CN `www.example.com`); en Azure lo emite Let's Encrypt, y eso no se pudo verificar aquí.

4. Tramo interno: WAF -> aplicación

BunkerWeb termina el TLS y reenvía en HTTP por la red `app` (`web:8000`).

Medida	Valor
Peticiones HTTP WAF->web durante el recorrido	18
Tramas WAF->web con el centinela (control positivo, esperado > 0)	2
Tramas WAF->GLPI con el centinela	**0**
Cabeceras de seguridad que salen del upstream (CSP, X-Frame-Options, X-Request-ID)	19

Cookies de respuesta observadas (valores enmascarados): `sessionid` con `HttpOnly; Secure; SameSite=Lax`; `csrftoken` con `HttpOnly; Secure; SameSite=Lax; Max-Age=31449600`. Que el centinela aparezca en este tramo es **esperado**: el WAF termina el TLS y reenvía el cuerpo del formulario. Lo relevante es que no sale de la red `app` ni llega a GLPI. El límite de confianza es el equipo, no el nombre de la red.

5. Aplicación ? PostgreSQL

Medida	Valor
Tramas	3 150
`SSLRequest` y conexiones con TLS	0 y 0 (PostgreSQL local sin TLS; Supabase usa `verify-full`)
Autenticación SCRAM vista / contraseña en claro	21 / **0**
`BEGIN` / `COMMIT`	127 / 127
Tramas con el centinela hacia la base (control positivo)	36

El centinela viaja a PostgreSQL porque ahí **debe** guardarse (`solicitud_restringida`). Que esté es la prueba de que la captura ve; que no esté en GLPI ni en el tramo externo es el resultado. Cada transacción abre y cierra (127 de 127): no quedan transacciones colgadas.

6. Un matiz que la versión anterior no veía: el enlace de parámetros

La Entrega 3 dijo que el tráfico mostraba «sentencias con identificadores entre comillas y valores ligados». Medido ahora: **0 mensajes** `Parse` **y 0** `Bind`**, 1 332 consultas simples**. Django 5.2 con `psycopg` 3 enlaza los parámetros **del lado del cliente**: `psycopg` escapa cada valor y arma el texto de la consulta antes de enviarla. Consecuencias:

- **No hay inyección**: el escape es del controlador, y el código solo usa el ORM o `cursor.execute(sql, params)`; `bandit`, las reglas `S` de `ruff` y sqlmap hasta nivel 5 coinciden.
- **El valor viaja dentro del texto SQL.** Si la base registrara sentencias (`log_statement`, o el texto de una consulta fallida), el campo restringido aparecería en esos registros. Con enlace en el servidor aparecería como `$1`. Es una razón para la opción `server_side_binding` de Django, a evaluar frente a su compatibilidad con el *session pooler* de Supabase. No se abrió hallazgo: no hay una fuga medida, solo una condición.

7. Worker ? GLPI: los tokens viajan en claro (N-1)

El único cliente de GLPI es el worker, por HTTP en la red `app` (`http://glpi/apirest.php`).

Medida (31 240 tramas)	Valor
Peticiones con `App-Token` en claro	**382**
Peticiones con `Session-Token` en claro	**378**
Peticiones con `user_token` en `Authorization`	**4**
Tramas con el centinela de T7	**0**

La Entrega 3 midió 130, 129 y 1, con otro volumen de flujos; la **conducta es la misma**. Quien comparta la red `app` (`web`, `bunkerweb`, `glpi` y `worker`) y se comprometa puede leer esas cabeceras y actuar con los permisos del worker. `web` nunca llama a GLPI (invariante, probado en `test_caidas.py`), pero puede espiar. **Hallazgo B-32 (Medio)**; propuesta P-1: una red `glpi_api` que compartan solo `worker` y `glpi`. A esto se suma que el perfil de la cuenta de servicio sobra en derechos (**B-35**: puede borrar y purgar tickets).

8. GLPI ? MariaDB (V-04)

30 133 tramas, **ninguna con TLS**, 11 051 consultas legibles. La red `db_net` es `internal: true` y solo la comparten GLPI y MariaDB; el usuario `glpi` no tiene `FILE`. Riesgo Bajo, pero dos medidas baratas: `local_infile=0` y, si se acepta P-1, no hace falta TLS interno. **B-30**.

9. Comparación con la Entrega 3

Aspecto	Entrega 3 (24/10)	Esta auditoría (24–25/10)
Puntos de captura	`bunkerweb`, `postgres`, `glpi`	los mismos
Tramas (bunkerweb / postgres / glpi)	3 264 / 6 537 / 15 093	821 / 3 150 / 31 240
TLS externo	1.3, suite `0x1302`	1.3, suite `0x1302`
Centinela hacia GLPI	0	0
Centinela en el tramo externo	0 (cifrado)	0
Tokens de GLPI en claro	130 / 129 / 1 · severidad Baja (C-01)	382 / 378 / 4 · **Media** (B-32, con DREAD)
Consultas	«valores ligados»	enlace del lado del cliente (§6)
Aserciones automáticas	no	sí; el script falla si una se rompe
Reproducible con un comando	parcial	sí

La diferencia de severidad no es de medición sino de método: aquí cada hallazgo lleva DREAD, un factor por la posición del atacante y una prueba que se rompe al quitar el control.

10. Límites

- El stack local no cifra PostgreSQL ni MariaDB; el tramo real hacia Supabase (`verify-full`) **no se capturó** (Supabase en solo lectura de catálogo).
- Se capturó dentro del equipo, no en una interfaz física: el tramo de internet no existe en el laboratorio.
- En la segunda corrida del recorrido Playwright, un caso (RET) recibió un 429 del WAF; las aserciones de tráfico son válidas, pero el recorrido no fue 4 de 4. Ver **B-36**.